

University of Asia Pacific

Department of Computer Science and Engineering

CSE 450: VLSI Lab

LAB REPORT

Experiment Number: 05

Experiment Title: Design and Simulation of a 4-Bit Register with Asynchronous Reset and Write Enable Using VHDL

Submitted by:

Name: Nazmul Mridha
Student ID: 22201161
Section: C2

Submitted to:

Md. Rasheduzzaman
Lecturer
Department of Computer Science and Engineering

Date of Submission: 29/10/2026

1. Experiment Name

Design and Simulation of a 4-Bit Register with Asynchronous Active-Low Reset and Synchronous Write Enable Using VHDL in Xilinx ISE 14.7.

2. Objective

The main objectives of this experiment are:

1. To understand the working principle of a register as the basic storage element used in processors and digital systems.
2. To design a 4-bit register in VHDL with a clock (CLK), an asynchronous active-low reset (RST_N), a synchronous write enable (WE), a 4-bit data input (DIN) and a 4-bit data output (DOUT).
3. To describe the register with a clocked process so that the synthesizer infers edge-triggered flip-flops.
4. To write a self-checking testbench that verifies reset, write, hold and reset-priority behaviour.
5. To generate the RTL schematic of the designed circuit in Xilinx ISE 14.7.
6. To simulate the design in ISim and examine the output for different input combinations.
7. To verify the functionality of the design using the register function table.

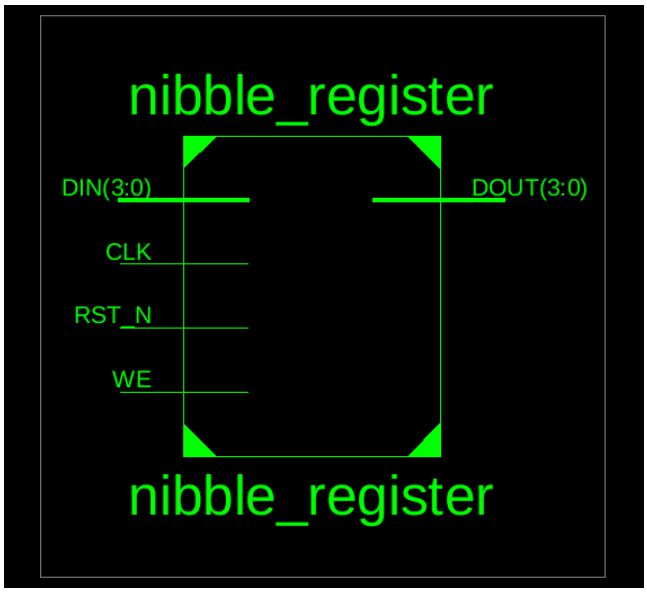
Design Description

The nibble_register is a sequential circuit that stores a 4-bit value. It is described in RTL style using a single process that is sensitive to CLK and RST_N. When RST_N is low, the stored value is cleared to 0000 immediately, without waiting for a clock edge (asynchronous reset), and the reset has priority over every other input. When RST_N is high, the value on DIN is loaded into the register on the rising edge of CLK only if WE is 1. If WE is 0, the register keeps its old value, so changes on DIN are ignored. The internal signal reg_q holds the stored value and is connected to the output DOUT. The behaviour is summarized in the function table below.

RST_N	WE	CLK	DIN	DOUT (next value)
0	X	X	XXXX	0000 (async clear)
1	0	Rising edge	XXXX	Hold (no change)
1	1	Rising edge	D3 D2 D1 D0	D3 D2 D1 D0 (load)
1	X	No edge	XXXX	Hold (no change)

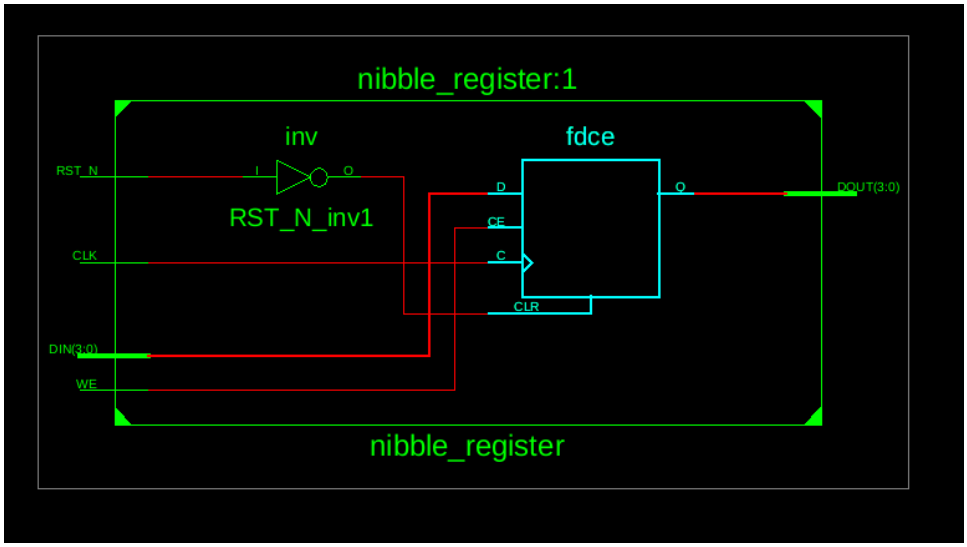
Table: Function table of the 4-bit register (X = don't care)

3. RTL-Schematic Diagrams



Top-Level RTL Schematic of the 4-Bit Register (nibble_register)

The top-level schematic shows the ports of the design: the clock input CLK, the active-low reset input RST_N, the write-enable input WE, the 4-bit data input DIN(3:0) and the 4-bit data output DOUT(3:0).



Internal RTL Schematic of the 4-Bit Register

The internal schematic shows how the synthesizer implemented the register. DIN(3:0) is connected to the D inputs of the fdce flip-flops, WE is connected to the clock-enable input CE, and CLK drives the clock input C. The fdce cell is a D flip-flop with clock enable and an asynchronous clear (CLR). Because CLR is active-high while RST_N is active-low, an inverter cell (RST_N_inv1) is placed between RST_N and CLR. The Q outputs drive DOUT(3:0). The design contains only the

flip-flops and one inverter, with no extra combinational logic, which confirms that the write enable is implemented by the flip-flop’s own clock-enable pin.

4. Code in github repository

https://github.com/Nazmul-SWE/VLSI

Design code (nibble_register.vhd):

```
--------------------------------------------------------------------------------
-- Design  : 4-bit register
-- Features: asynchronous active-low reset (RST_N), synchronous write enable (WE)
--------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity nibble_register is
    Port ( CLK   : in  STD_LOGIC;                          -- clock, rising edge
           RST_N : in  STD_LOGIC;                          -- asynchronous reset, active-low
           WE    : in  STD_LOGIC;                          -- write enable
           DIN   : in  STD_LOGIC_VECTOR (3 downto 0);      -- data in
           DOUT  : out STD_LOGIC_VECTOR (3 downto 0) );    -- stored data out
end nibble_register;

architecture RTL of nibble_register is
    signal reg_q : STD_LOGIC_VECTOR (3 downto 0);
begin

    reg_proc : process (CLK, RST_N)
    begin
        if RST_N = '0' then
            reg_q <= "0000";                    -- reset acts immediately
        elsif rising_edge(CLK) then
            if WE = '1' then
                reg_q <= DIN;                   -- write
            end if;                             -- otherwise keep old value
        end if;
    end process reg_proc;

    DOUT <= reg_q;

end RTL;
```

Testbench code (nibble_register_tb.vhd):

```
--------------------------------------------------------------------------------
-- Testbench for nibble_register (4-bit register)
--------------------------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity nibble_register_tb is
end nibble_register_tb;

architecture sim of nibble_register_tb is

    constant CLK_HALF : time := 5 ns;                   -- 10 ns clock period

    signal clk   : std_logic := '0';
    signal rst_n : std_logic := '0';
    signal we    : std_logic := '0';
    signal din   : std_logic_vector(3 downto 0) := "0000";
    signal dout  : std_logic_vector(3 downto 0);
    signal done  : boolean := false;
```

```
    type nibble_array is array (0 to 5) of std_logic_vector(3 downto 0);
    constant TEST_DATA : nibble_array := ("1010", "0101", "0001", "1000", "1111", "0110");

begin

    dut : entity work.nibble_register
        port map ( CLK => clk, RST_N => rst_n, WE => we, DIN => din, DOUT => dout );

    -- clock runs until the stimulus is finished
    clock_gen : process
    begin
        while not done loop
            clk <= '0'; wait for CLK_HALF;
            clk <= '1'; wait for CLK_HALF;
        end loop;
        wait;
    end process clock_gen;

    stimulus : process

        -- apply one clock cycle and check the output just after the edge
        procedure cycle (value      : in std_logic_vector(3 downto 0);
                         write_en   : in std_logic;
                         expect_out : in std_logic_vector(3 downto 0);
                         msg        : in string) is
        begin
            din <= value;
            we  <= write_en;
            wait until rising_edge(clk);
            wait for 1 ns;
            assert dout = expect_out
                report "FAIL: " & msg severity error;
        end procedure;

    begin
        -- 1. power-up: reset held low
        rst_n <= '0';
        wait for 12 ns;
        assert dout = "0000" report "FAIL: async reset did not clear output" severity
error;

        -- release reset away from a clock edge
        rst_n <= '1';
        wait until falling_edge(clk);

        -- 2. WE = 0 : input must be ignored
        cycle("1111", '0', "0000", "write happened with WE=0");

        -- 3. write every pattern in the table
        for i in TEST_DATA'range loop
            cycle(TEST_DATA(i), '1', TEST_DATA(i), "pattern not stored");
        end loop;

        -- 4. hold: new data on DIN but WE = 0 (last stored value was "0110")
        cycle("1001", '0', "0110", "value changed with WE=0");
        cycle("0000", '0', "0110", "value changed with WE=0 (second hold)");

        -- 5. asynchronous reset in the middle of a clock cycle
        wait until falling_edge(clk);
        rst_n <= '0';
        wait for 1 ns;                          -- no clock edge has happened yet
```

```
        assert dout = "0000" report "FAIL: reset is not asynchronous" severity error;
        wait until rising_edge(clk);
        wait for 1 ns;
        assert dout = "0000" report "FAIL: output changed while reset active" severity
error;

        -- 6. writes are ignored during reset, then work again after release
        cycle("1100", '1', "0000", "write accepted during reset");
        wait until falling_edge(clk);
        rst_n <= '1';
        cycle("1100", '1', "1100", "write failed after reset release");

        report "PASS: nibble_register testbench completed" severity note;
        done <= true;
        wait;
    end process stimulus;

end sim;
```

5. Simulations

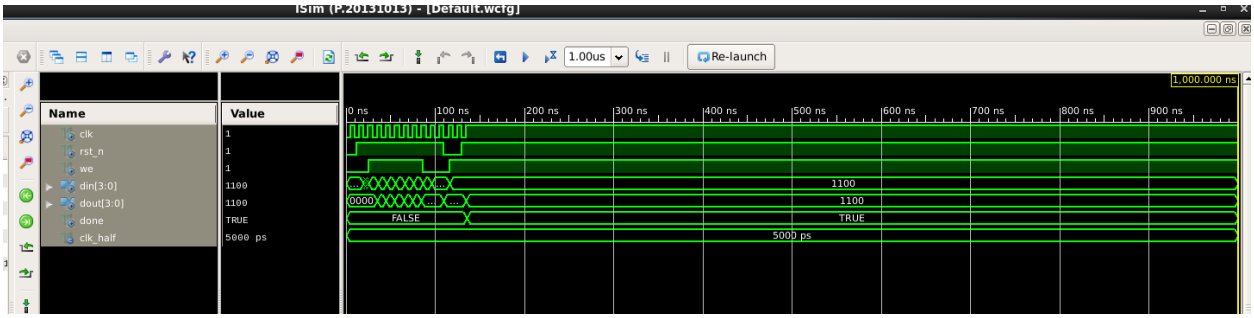


Figure: Design Simulation Waveform (ISim)

The testbench generates a 10 ns clock (5 ns high, 5 ns low) and applies the stimulus around the clock edges. After each rising edge it waits 1 ns and compares DOUT with the expected value using assert statements; any mismatch is reported as an error message, and at the end the testbench prints a PASS message. In total 14 checks are performed in the sequence below. The simulation ends at about 136 ns, when the done flag becomes TRUE and the clock generator stops, which is why the clock stays high in the waveform afterwards.

The test sequence is as follows. First, RST_N is held low at power-up and DOUT is checked to be 0000, then reset is released away from a clock edge. Next, DIN = 1111 is applied with WE = 0 and must be ignored. Six patterns (1010, 0101, 0001, 1000, 1111, 0110) are then written with WE = 1. Two hold checks follow with new data on DIN but WE = 0. After that, RST_N is pulled low in the middle of a clock cycle to show that the reset is asynchronous, and a write attempt during reset is shown to be ignored. Finally, reset is released and the value 1100 is written successfully. The expected results of the testbench, which match the simulated waveform, are listed below.

#	Time	Test	RST_N	WE	DIN	Expected DOUT
1	12 ns	Asynchronous reset at power-up	0	0	0000	0000
2	26 ns	Write ignored when WE = 0	1	0	1111	0000
3	36 ns	Write pattern 1	1	1	1010	1010
4	46 ns	Write pattern 2	1	1	0101	0101
5	56 ns	Write pattern 3	1	1	0001	0001
6	66 ns	Write pattern 4	1	1	1000	1000
7	76 ns	Write pattern 5	1	1	1111	1111
8	86 ns	Write pattern 6	1	1	0110	0110
9	96 ns	Hold (new DIN, WE = 0)	1	0	1001	0110
10	106 ns	Hold (second check)	1	0	0000	0110
11	111 ns	Reset in mid-cycle, before any clock edge	0	0	0000	0000
12	116 ns	Output stays 0000 across clock edge	0	0	0000	0000
13	126 ns	Write ignored during reset	0	1	1100	0000
14	136 ns	Write works after reset release	1	1	1100	1100

Table: Register Test Sequence and Expected Output

The simulated waveform follows the function table: DOUT is cleared as soon as RST_N goes low, without waiting for a clock edge. DIN is loaded into DOUT on the rising clock edge only when WE is 1, DOUT holds its value when WE is 0, and reset takes priority over a write. The final value of DOUT in the waveform is 1100, as expected.

6. Conclusion

In this experiment, a 4-bit register with an asynchronous active-low reset and a synchronous write enable was designed in VHDL using Xilinx ISE 14.7. The RTL schematic shows that the design is implemented with fdce flip-flops (D flip-flops with clock enable and asynchronous clear) and a single inverter for the active-low reset. Behavioral simulation in ISim with a self-checking testbench confirmed that the register clears immediately on reset, loads DIN only on a rising clock edge when WE is 1, holds its value when WE is 0, and ignores writes while reset is active. Since the simulated outputs match the expected function table, the design is considered functionally verified. This experiment provided practical experience in sequential circuit design, clocked processes in VHDL, self-checking testbenches and digital logic simulation, and registers are a fundamental building block of processors and computing systems.